

CSE 331

AUTOMATA AND COMPUTABILITY

Lecture 10 (DFA to RE)

Summer 2026

Today's learning goals

Sipser Ch 1.2, 1.3

- Convert between DFA to Regular expression



GNFA (Generalizing NFA)

$$\Sigma = \{a, b\}$$

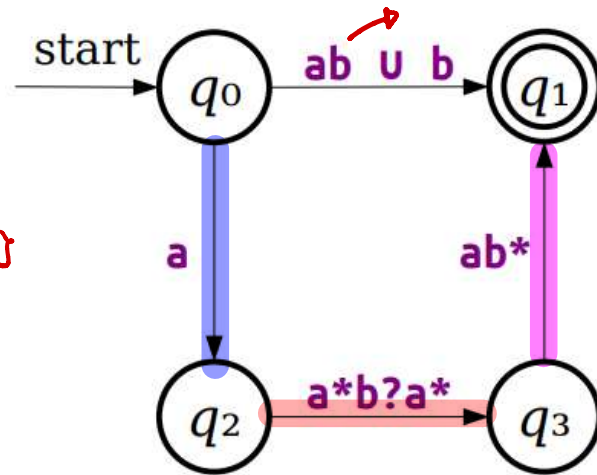
- GNFA: NFA with **regular expressions** on edges

$$L = \{a\}$$

$$a^* = \text{zero or more } a\text{'s}$$

$$a^* = \{\epsilon, a, aa, aqa \dots\}$$

$$L^* = L^0 \cup L^1 \cup L^2 \dots$$



$$ab \cup b$$

$$\{ab, b\}$$

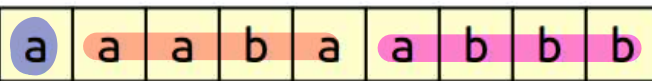
$$a^*b^?a^*$$

$$a^*(b \cup \epsilon)a^*$$

$$\{\epsilon, a, aa, \dots\} \{b, \epsilon\} \{\epsilon, a, aa, \dots\}$$

$$\{b, ab, aab, \epsilon, a, aa, \dots\} \{\epsilon, a, aa, \dots\}$$

$$\{b, ba, baq, ab, abq, abqq, aqb, aqbq, aqbqa \dots\}$$



Generalizing NFAs

- **Key Idea 1:** Imagine that we can label transitions in an NFA with arbitrary regular expressions.
- **Key Idea 2:** If we can convert an NFA into a generalized NFA that looks like this...



- ...then we can easily read off a regular expression for the original NFA.

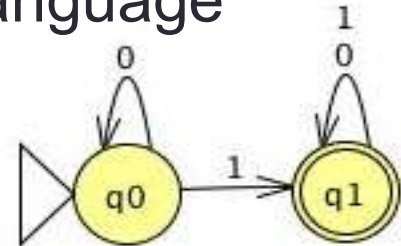
DFA to regexp

Lemma 1.60, page 69

- Idea: use intermediate model **GNFA** whose labels are regular expressions

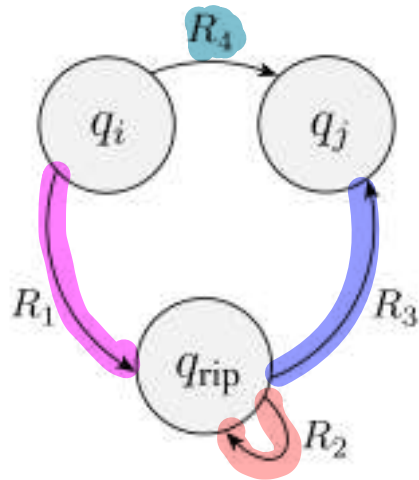


- E.g.: build regular expression describing language recognized by



GNFA simplification (How to eliminate a state)

- Simplify GNFA by **removing one node at a time**

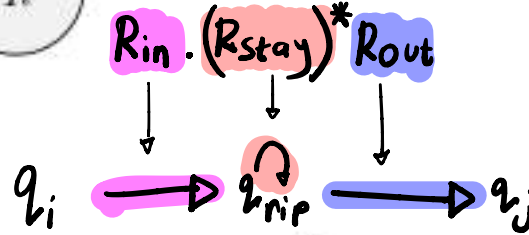
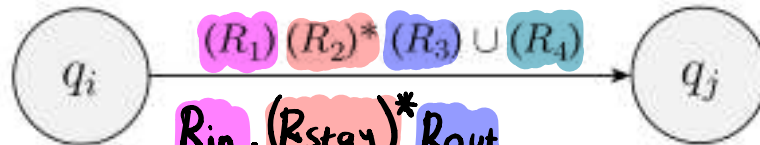


before

Eliminate q_{rip} :

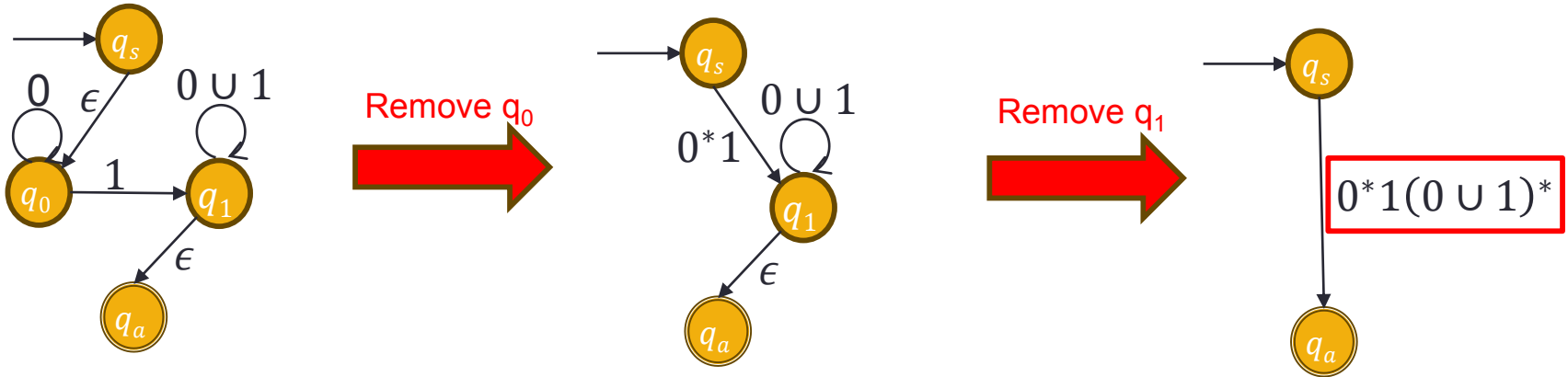
IN	out
q_i	q_j

$q_i \longrightarrow q_j$



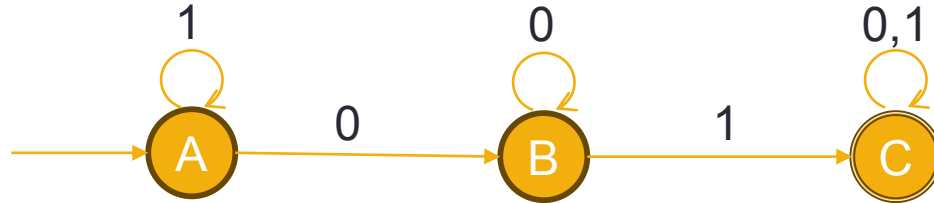
after

GNFA simplification



DFA to regexp exercise

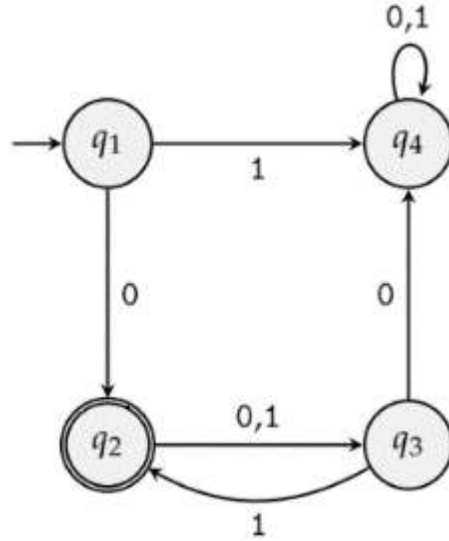
- Convert the following DFA to regular expression via GNFA



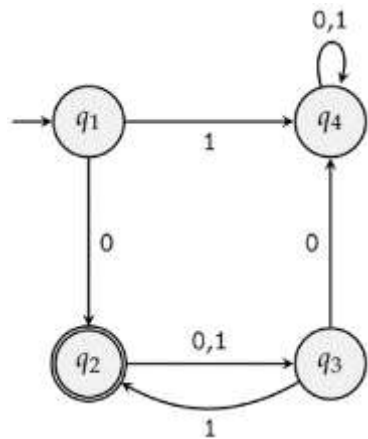
DFA to regexp exercise

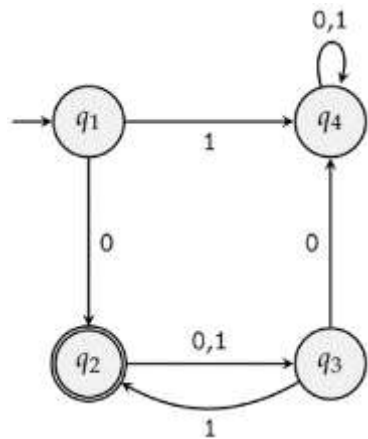
- Convert the following DFA into an equivalent regular expression using the state elimination method. First eliminate q_2 , then q_3 and finally q_4 . You must show work.

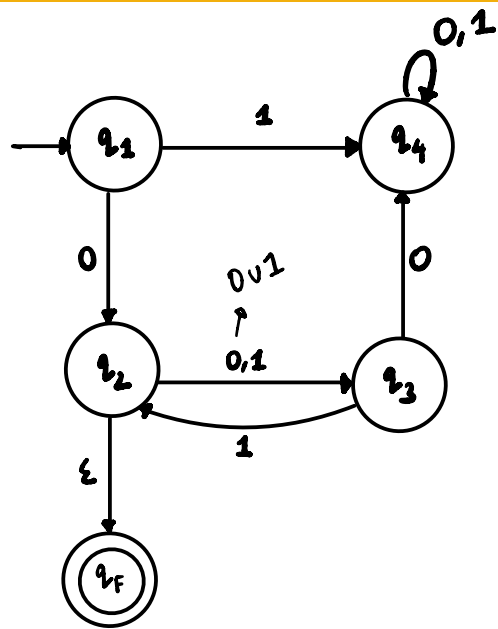
If Start State has incoming transition, then you need to Create a new start state.



You always need to Create a new final state.







Eliminate q_2, q_3, q_4

Eliminate q_2

In	Out
q_1	q_3
q_3	q_F

$$\begin{array}{c} 0 \cup 1 \\ \downarrow \quad \downarrow \\ \{0\} \cup \{1\} = \{0, 1\} \end{array}$$

$$R_{in} \circ R_{start}^* \circ R_{out}$$

$$R_{in} \circ R_{start}^* \circ R_{out}$$

$$q_1 \xrightarrow{0} q_2 \xrightarrow{(0 \cup 1)} q_3 =$$

$$0 \cdot (0 \cup 1)$$

$$q_1 \xrightarrow{0} q_2 \xrightarrow{\epsilon} q_F =$$

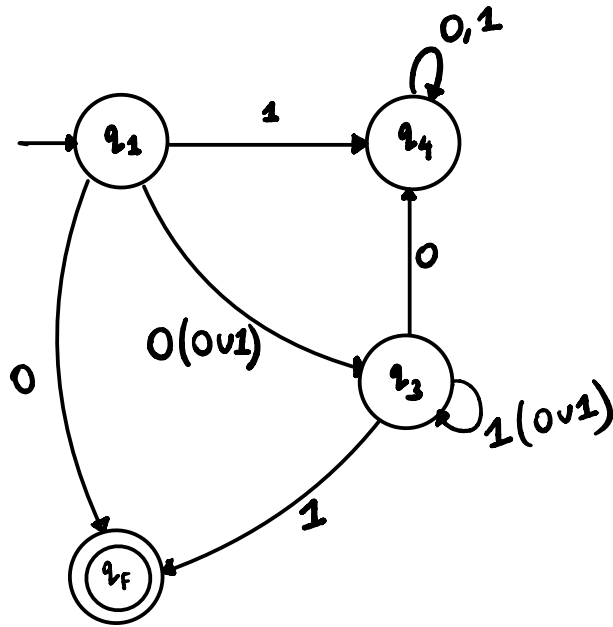
$$0 \cdot \epsilon = 0$$

$$q_3 \xrightarrow{1} q_2 \xrightarrow{(0 \cup 1)} q_3 =$$

$$1 \cdot (0 \cup 1)$$

$$q_3 \xrightarrow{1} q_2 \xrightarrow{\epsilon} q_F =$$

$$1 \cdot \epsilon = 1$$



Eliminate ~~q2~~, q3, q4

Eliminate q3

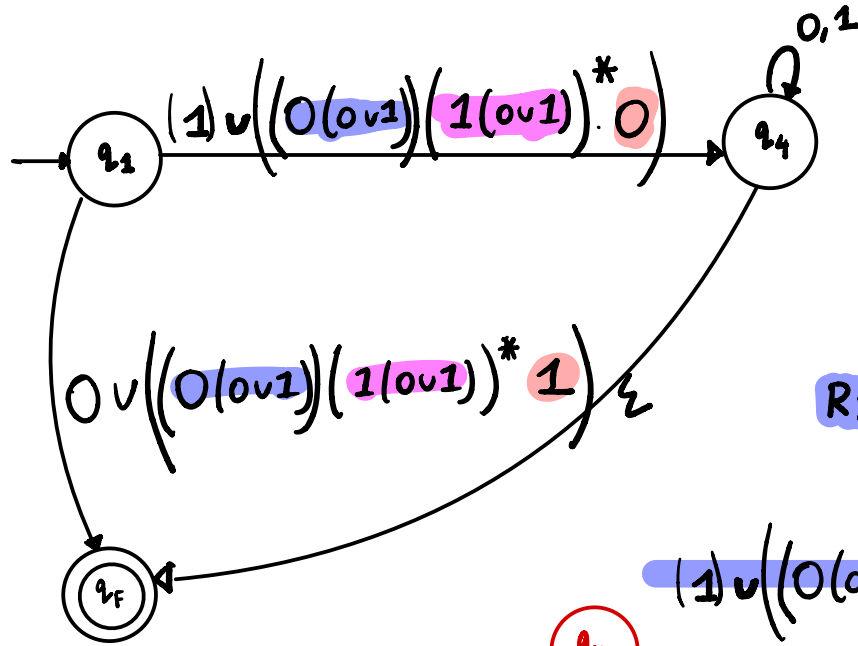
In	Out
q1	q4
	qF

$$R_{in} \circ R_{stay}^* \circ R_{out}$$

$$R_{in} \circ R_{stay}^* \circ R_{out}$$

$$q_1 \xrightarrow{0(ov1)} q_3 \xrightarrow{1(ov1)} q_4 \xrightarrow{0} q_4 = (0(ov1))(1(ov1))^* 0$$

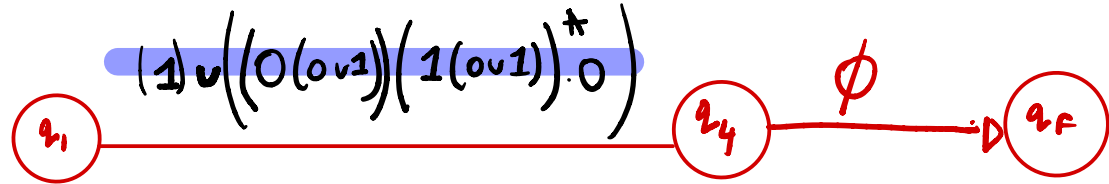
$$q_1 \xrightarrow{0(ov1)} q_3 \xrightarrow{1(ov1)} q_3 \xrightarrow{1} q_F = (0(ov1))(1(ov1))^* 1$$



Eliminate q_4

In	Out
q_1	q_F

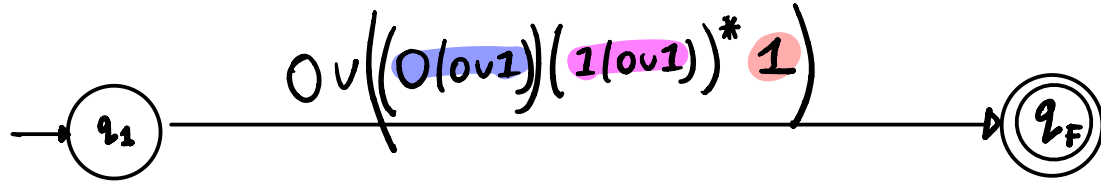
$$R_{in} \circ R_{start}^* \circ R_{out}$$



$$R_{in} \cdot R_{out} = R_{in} \cdot \emptyset = \emptyset$$

q_4 is a dead state, since it cannot reach final state.

Eliminate ~~q_2~~ , ~~q_3~~ , ~~q_4~~

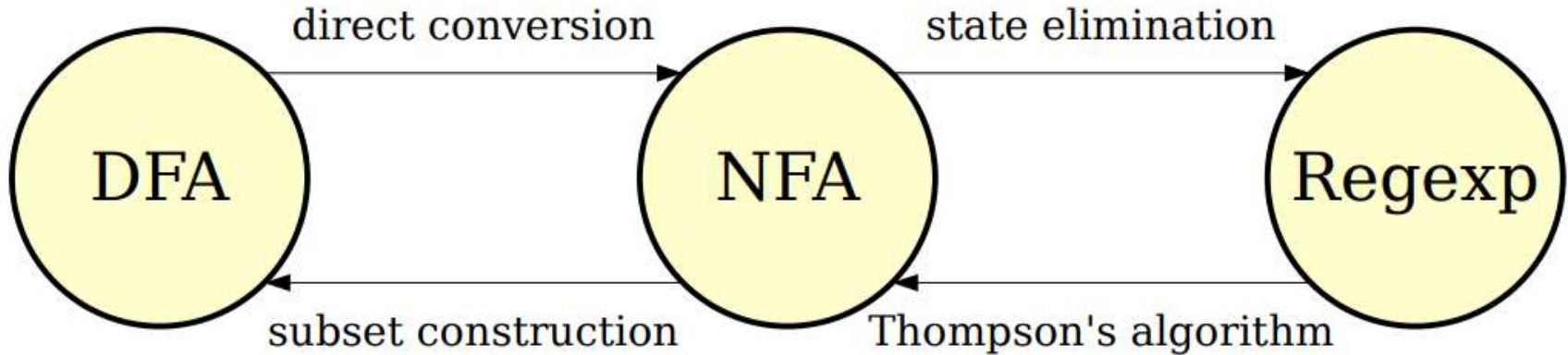


Final expression: $0v((0|0v1)(1|0v1)^*1)$

Eliminate ~~q_2~~ , ~~q_3~~ , ~~q_4~~

Our Transformations

Regular language



The State-Elimination Algorithm

- Start with an NFA N for the language L .
- Add a **new start state** q_s (if there is an incoming transition to the original start state) and **accept state** q_f to the NFA.
- Add an **ϵ -transition** from q_s to the **old start state** of N .
- Add **ϵ -transitions** from **each accepting state** of N to q_f , then mark them as not accepting.
- Repeatedly **remove states other than q_s and q_f** from the NFA by “shortcutting” them until only two states remain: q_s and q_f .
- The transition from q_s to q_f is then a regular expression for the NFA.



The State-Elimination Algorithm

- To **eliminate a state** q from the automaton, do the following for each pair of states q_0 and q_1 , where there's a transition from q_0 into q and a transition from q into q_1 :
 - Let R_{in} be the regex on the transition from q_0 to q .
 - Let R_{out} be the regex on the transition from q to q_1 .
 - If there is a regular expression R_{stay} on a transition from q to itself, add a new transition from q_0 to q_1 labeled $((R_{in})(R_{stay})^*(R_{out}))$.
 - If there isn't, add a new transition from q_0 to q_1 labeled $((R_{in})(R_{out}))$
- If a pair of states has multiple transitions between them labeled $R_1, R_2, \dots, R_k$, replace them with a single transition labeled $R_1 \cup R_2 \cup \dots \cup R_k$.

All roads lead to ... regular sets?

Are there any languages over $\{0,1\}$ that are not regular?

- ☒ A. Yes: a language that is recognized by an NFA but not any DFA.
- ☒ B. Yes: there is some infinite language of strings over $\{0,1\}$ that is not described by any regular expression.
- ☒ C. No: all languages over $\{0,1\}$ are regular because that's what it means to be a language.
- ☒ D. No: for each set of strings over $\{0,1\}$, some DFA recognizes that set.
- ☒ E. I don't know.

Bounds on DFA

$$a^n b^n$$

- in DFA, memory = states
- Automata can only "remember" ...
 - ...finitely far in the past
 - ...finitely much information
- If a computation path visits the same state more than once, the machine can't tell the difference between the first time and future times it visited that state.